

AI Business Requirements Assistant

An LLM-based tool for generating structured BRDs and user stories from raw stakeholder input

Prepared by Naji | artificial intelligence | Business Analyst Track

1. Business Problem

Business Analysts spend a significant portion of their time converting unstructured stakeholder input — interview transcripts, meeting notes, email threads — into structured deliverables: Business Requirements Documents (BRDs), user stories, and scope definitions. This process is manual, slow, and prone to two specific failure modes: (1) genuine ambiguities or stakeholder disagreements get silently resolved by the analyst's best guess rather than being flagged for clarification, and (2) requirements documents often include boilerplate assumptions (security, performance, compliance specifications) that were never actually discussed with the stakeholder, presented with false confidence.

This project builds an AI assistant that takes a raw stakeholder transcript and generates a structured requirements draft — while explicitly avoiding both failure modes above. The goal was not simply to summarize a transcript, but to produce a draft that a real Business Analyst could use as a defensible starting point: one that clearly separates what stakeholders actually said from what still needs to be clarified.

2. Approach

The system takes a single-pass LLM approach rather than a multi-step pipeline, since the task is extraction and structuring rather than retrieval — no external knowledge base is needed, unlike a RAG system. The core engineering work is in the prompt design itself.

Stage	Tool	Output
Input	Raw stakeholder transcript (interview, meeting notes)	Unstructured text
Prompt design	Groq API (openai/gpt-oss-120b) with grounding-enforced prompt	Structured extraction instructions
Generation	LLM call via Groq	BRD sections, user stories, open questions
Interface	Streamlit	Paste-transcript-in, structured-document-out web app

2.1 Output Structure

Each generated document follows a consistent structure: Business Objective, In-Scope Items, Out-of-Scope Items, Functional Requirements, Non-Functional Requirements, Open Questions/Ambiguities, and User Stories (in standard “As a [role], I want [goal], so that [benefit]” format). This mirrors the structure of a real-world BRD closely enough to be immediately usable as a first draft.

2.2 Grounding Enforcement

The initial prompt version produced plausible-looking but fabricated Non-Functional Requirements — specific uptime percentages, encryption standards, and compliance frameworks that no stakeholder had mentioned. This was caught during testing and is documented in detail in Section 3. The corrected prompt explicitly instructs the model to omit any requirement category not discussed, or to state “Not discussed — recommend clarifying with stakeholder” rather than filling in an industry-standard default.

2.3 Disagreement Preservation

A second refinement, tested against a transcript with two stakeholders in active disagreement, instructed the model to explicitly surface conflicting positions in the Open Questions section — citing each stakeholder by name and position — rather than silently resolving the disagreement in favor of one party. This is a deliberate design choice: an AI tool that quietly picks a side in a real requirements conflict would actively mislead a BA rather than assist one.

3. Challenges and How They Were Solved

- **Fabricated Non-Functional Requirements:** the first prompt version generated a full, specific NFR section (99.5% uptime, AES-256 encryption, SOX compliance, 7-year retention) for a transcript that never discussed any of these topics. This is the most significant finding of the project: an ungrounded LLM will confidently invent industry-standard-sounding specifications rather than admit a gap. It was resolved by adding an explicit instruction to omit undiscussed categories or flag them, rather than by asking the model to “be careful” or “be accurate” in vaguer terms, which was tested first and did not reliably prevent the fabrication.
- **Silent resolution of stakeholder disagreement:** on a transcript with two stakeholders arguing for different priorities, an early prompt version produced a single, merged recommendation rather than surfacing the conflict. This mirrors a real and costly BA mistake — delivering a requirements document that looks decided when it is not. Resolved by explicitly instructing the model to name both stakeholders and their positions when disagreement is present in the source material.
- **Weak user story generation for system-level rules:** a small number of generated “user stories” described system behavior rather than genuine user goals (e.g., “As an employee, I want travel expenses excluded...”, which is a scope rule, not a user want). This is noted as a known limitation rather than fully resolved — addressing it would require a further prompt refinement distinguishing business rules from user-centered stories, which is a reasonable next iteration but was not necessary for the core deliverable.

4. Validation — Three Test Scenarios

The prompt was tested against three deliberately distinct transcripts, each designed to probe a different failure mode, rather than validating on a single easy case:

Test 1 — Single stakeholder, multiple unresolved details

Scenario: An expense-approval process interview where the stakeholder repeatedly second-guessed her own answers (escalation target, threshold ownership).

Result: The system correctly flagged both unresolved points as Open Questions rather than picking the stakeholder's last-mentioned preference as final, and correctly omitted all non-functional requirements the stakeholder never raised beyond the one she explicitly wanted (audit trail).

Test 2 — Two stakeholders in direct disagreement

Scenario: A ticket-tagging feature meeting where the Support Lead and Product Manager disagreed on whether urgency or category should be prioritized.

Result: The system explicitly cited both stakeholders by name and position in the Open Questions table, and attributed separate, appropriately distinct user stories to each stakeholder rather than merging their conflicting priorities into one artificial consensus.

Test 3 — Clean request with one buried ambiguity

Scenario: An HR onboarding-automation request that was mostly clear, with one genuine open question buried in an aside (whether contractors need a different checklist).

Result: The system correctly extracted the full functional scope, correctly identified the single genuine ambiguity, and kept the Non-Functional Requirements section limited to what could be reasonably inferred (data accuracy from the “real-time” requirement), explicitly marked as inferred rather than stated.

5. Tools Used

- Python — groq, streamlit
- Groq API (openai/gpt-oss-120b) — requirements extraction and generation
- Streamlit — web interface for pasting transcripts and generating drafts

- Anaconda / Jupyter Notebook — prompt development and testing environment

6. Repository and Deliverables

Project files are organized as follows:

- Documentation report (this document)
- Jupyter notebook covering prompt development and all three validation test runs
- app.py — Streamlit application
- Screenshots of the working application, including the onboarding-automation test case